



Sistemas Digitais - INE 5406/05235A - Grupo A
Universidade Federal de Santa Catarina

PROJETO E SÍNTESE DE UM MÓDULO CRIPTOGRÁFICO AES EM VHDL

Advanced Encryption Standard Project

Artur de Faria Rodrigo
Depto. De Ciencias da computação
Universidade Federal de Santa Catarina
Florianópolis, Santa Catarina, Brasil
arturdefariarodrigo@gmail.com

João Pedro Floriano
Depto. De Ciencias da computação
Universidade Federal de Santa Catarina
Florianópolis, Santa Catarina, Brasil
Joaopedroflorianodeoliveira@gmail.com

Nicolas de Brito Mafra
Depto. De Ciencias da computação
Universidade Federal de Santa Catarina
Florianópolis, Santa Catarina, Brasil
nicmafra2016@gmail.com

Pedro Zumstein Bilac
Depto. De Ciencias da computação
Universidade Federal de Santa Catarina
Florianópolis, Santa Catarina, Brasil
pedrozumstein@gmail.com

Thales Campos de Azevedo
Depto. De Ciencias da computação
Universidade Federal de Santa Catarina
Florianópolis, Santa Catarina, Brasil
thales4zev@gmail.com

Thásio Santos Silva
Depto. De Ciencias da computação
Universidade Federal de Santa Catarina
Florianópolis, Santa Catarina, Brasil
santosthasioo@gmail.com

Professor Ismael Seidel
Florianopolis, Santa Catarina, Brasil

Oque é AES?

- Sistema de criptografia simétrica dividido em blocos de 16 bytes
- Publicado pela FIPS (EUA), P197, em 2001
- Repetição de ciclos de transformações matemáticas, denominados rounds.
- O numero de rounds é diretamente proporcional ao tamanho da chave criptográfica selecionada, podendo ser de 128, 192 ou 256 bits.
- Rodadas: AES-128: 10, AES-192: 12, AES-256: 14
- Quatro etapas sequenciais: SubBytes, ShiftRows, MixColumns e AddRoundKey

Withdrawn NIST Technical Series Publication

Warning Notice

The attached publication has been withdrawn (archived), and is provided solely for historical purposes. It may have been superseded by another publication (indicated below).

Withdrawn Publication

Series/Number	Federal Information Processing Standards (FIPS) Publication 197
Title	Advanced Encryption Standard (AES)
Publication Date(s)	November 26, 2001
Withdrawal Date	May 9, 2023
Withdrawal Note	FIPS 197 is updated by NIST FIPS 197-upd1

Superseding Publication(s) (if applicable)

The attached publication has been **superseded by** the following publication(s):

Series/Number	NIST FIPS 197-upd1
Title	Advanced Encryption Standard (AES)
Author(s)	National Institute of Standards and Technology
Publication Date(s)	November 26, 2001; Updated May 9, 2023
URL/DOI	https://doi.org/10.6028/NIST.FIPS.197-upd1

Additional Information (if applicable)

Contact	Computer Security Division (Information Technology Laboratory)
Latest revision of the attached publication	
Related Information	This update makes no technical changes to the algorithm specified in the original (2001) release of this standard. This update includes extensive editorial improvements to the original version.
Withdrawal Announcement Link	https://csrc.nist.gov/News/2023/nist-updates-fips-197-advanced-encryption-standard

Razões para uso do AES

- **Padrão global:** Por ser um algoritmo gratuito, aberto e versátil, o sistema de criptografia AES é encontrado praticamente em todo lugar, como em conexões Wi-Fi, transferências bancárias e aplicativos de mensagem.
- **Alta segurança:** O Sistema AES usa uma forma extremamente robusta e eficiente de criptografar suas mensagens por meio de blocos de 16 bytes, tornando o dado quase indecifrável.

- **Alta velocidade em hardware:**

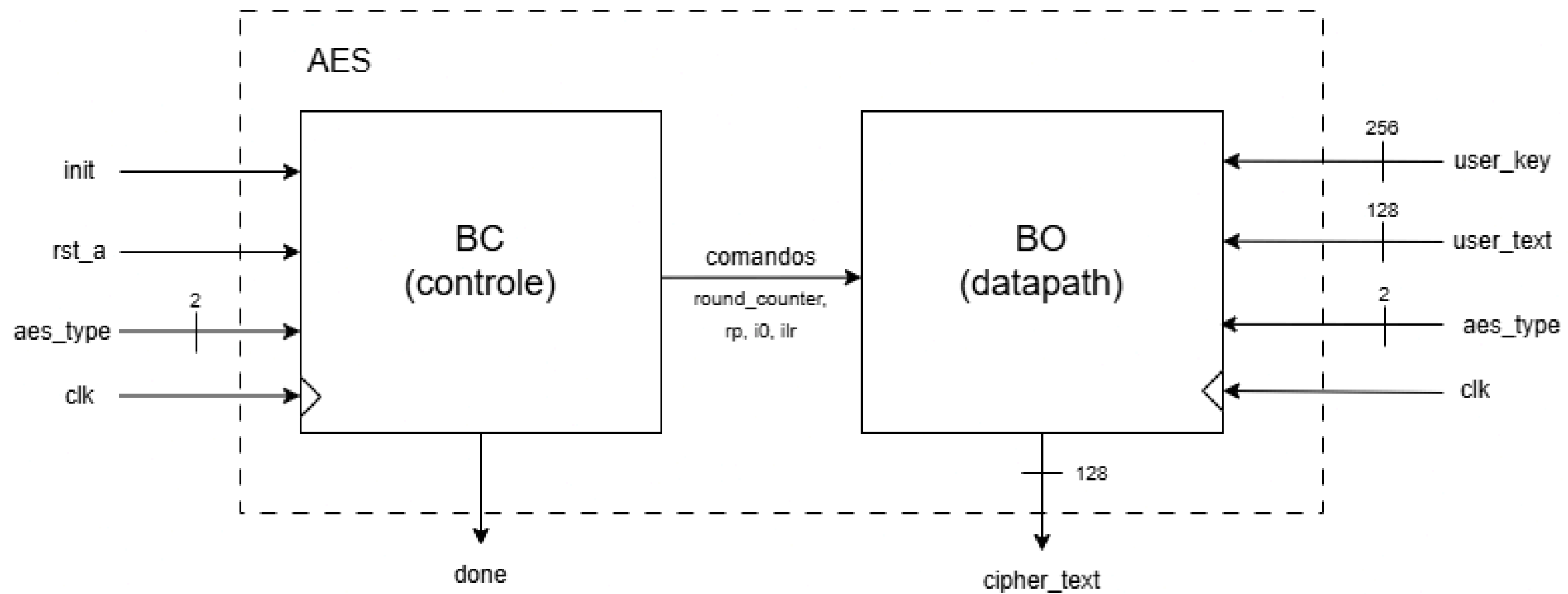
A criptografia realizada puramente via software necessita de respostas em tempo real, o que pode não ocorrer devido à latência. O AES, por possuir aceleração direta no hardware dos processadores modernos mitiga esse déficit, entregando respostas com uma velocidade incrível.

Diagrama Top-Level



AESType é um seletor que define com qual tamanho de chave o circuito irá trabalhar. Em caso da entrada ser “00”, será selecionado o AES-128; caso seja “01”, será selecionado o AES-192; e caso seja “10” será selecionado uma chave de AES-256 bits. Ele influencia diretamente na *user_key*, pois pode ignorar ou não os bits menos significativos (em tempo de execução, é impossível variar a largura da entrada). A diferença real entre tipos de AES consiste na quantidade de rodadas (rounds) que o AES irá executar, podendo ser de 10, 12 ou 14 ciclos, respectivamente, e consequentemente na quantidade de chaves de rodada geradas.

Diagrama BO/BC



BC (FSMD)

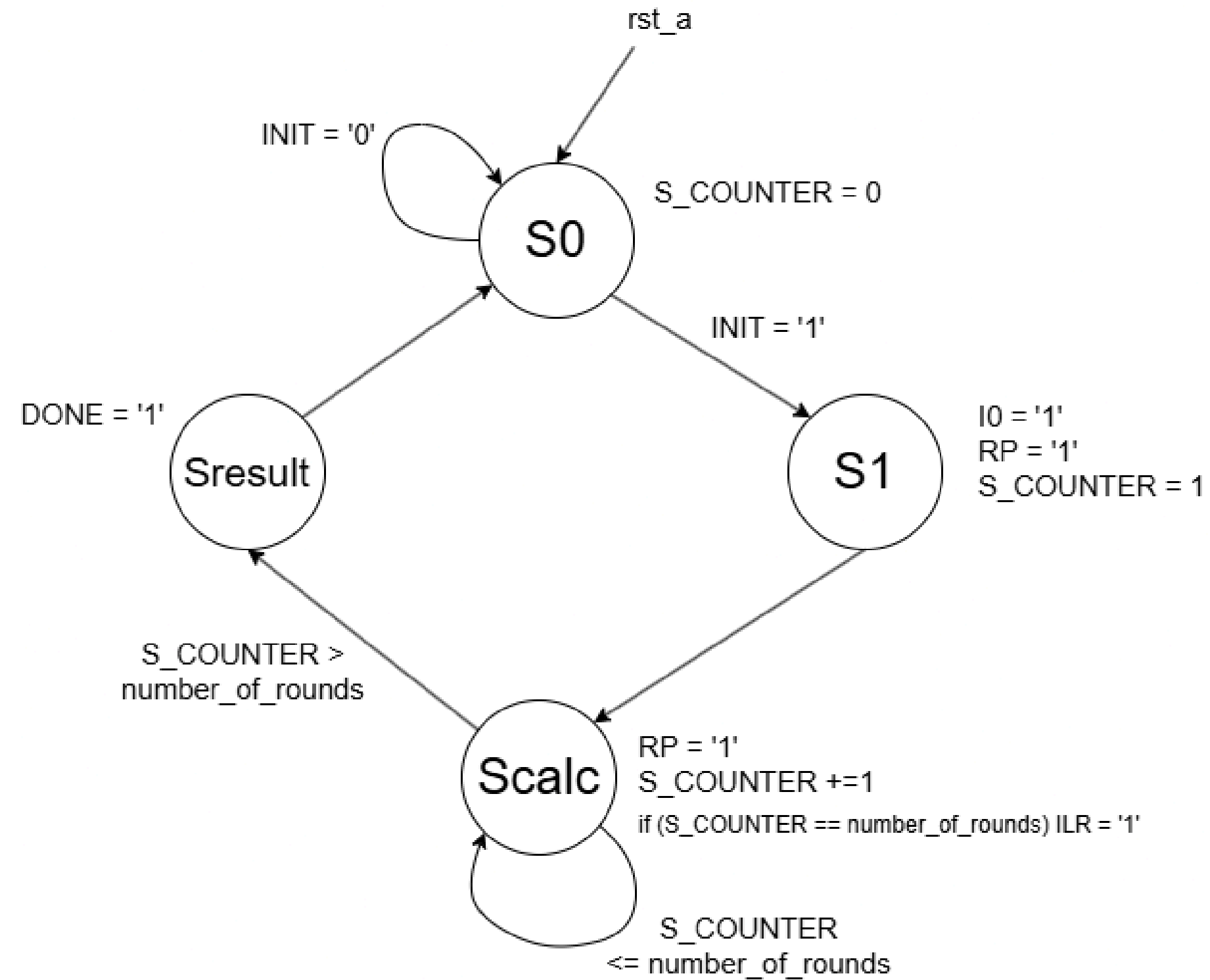
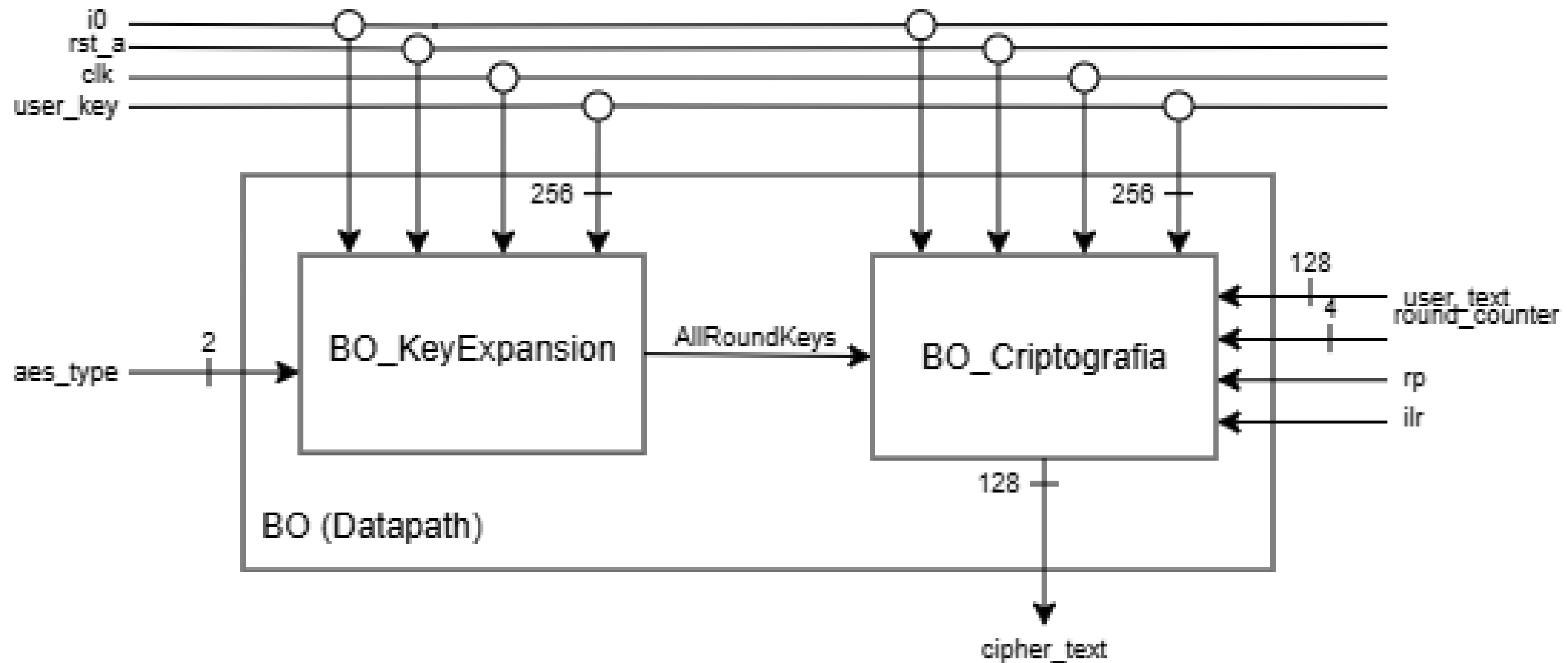
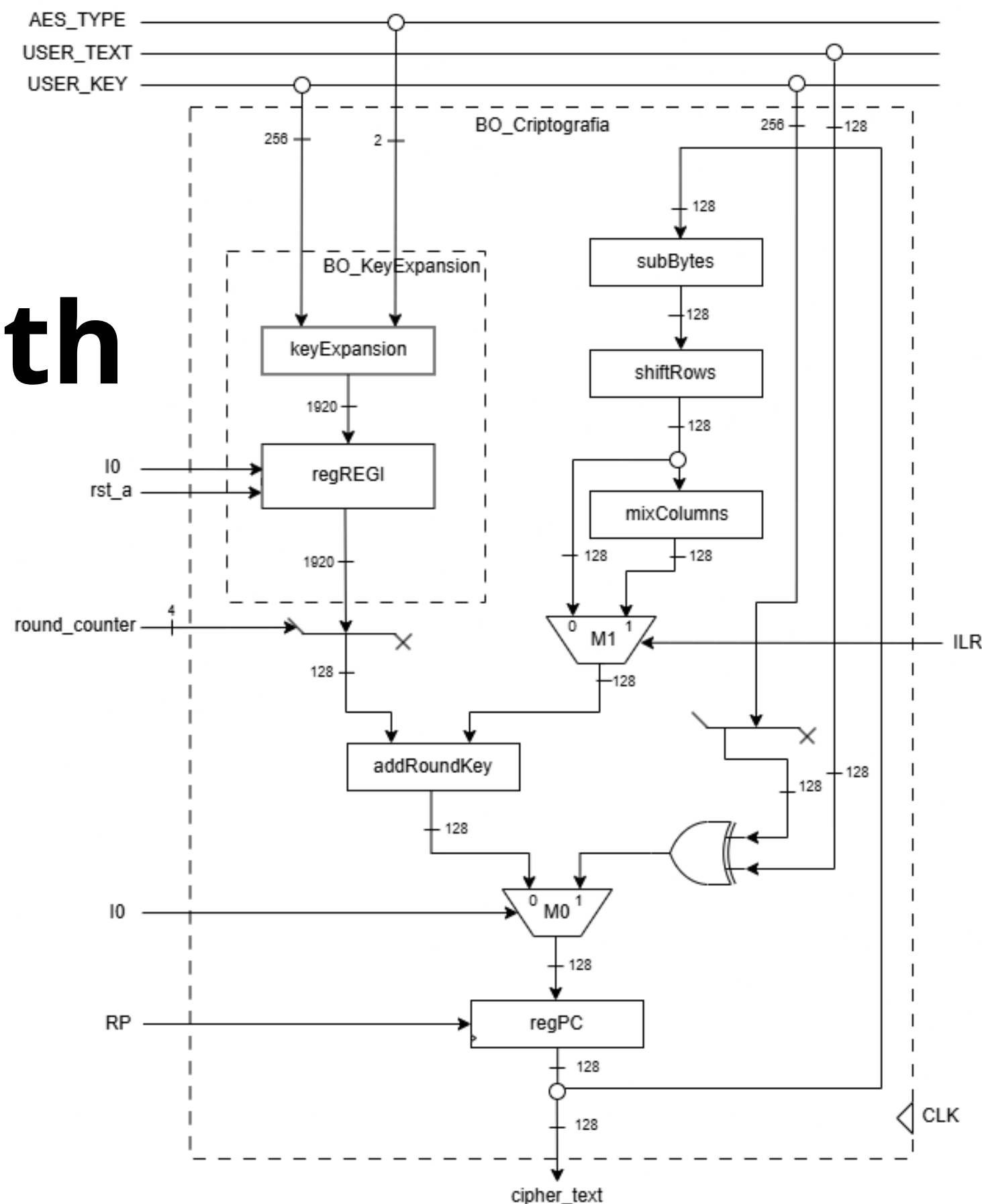


Diagrama Datapath (BO)



DataPath

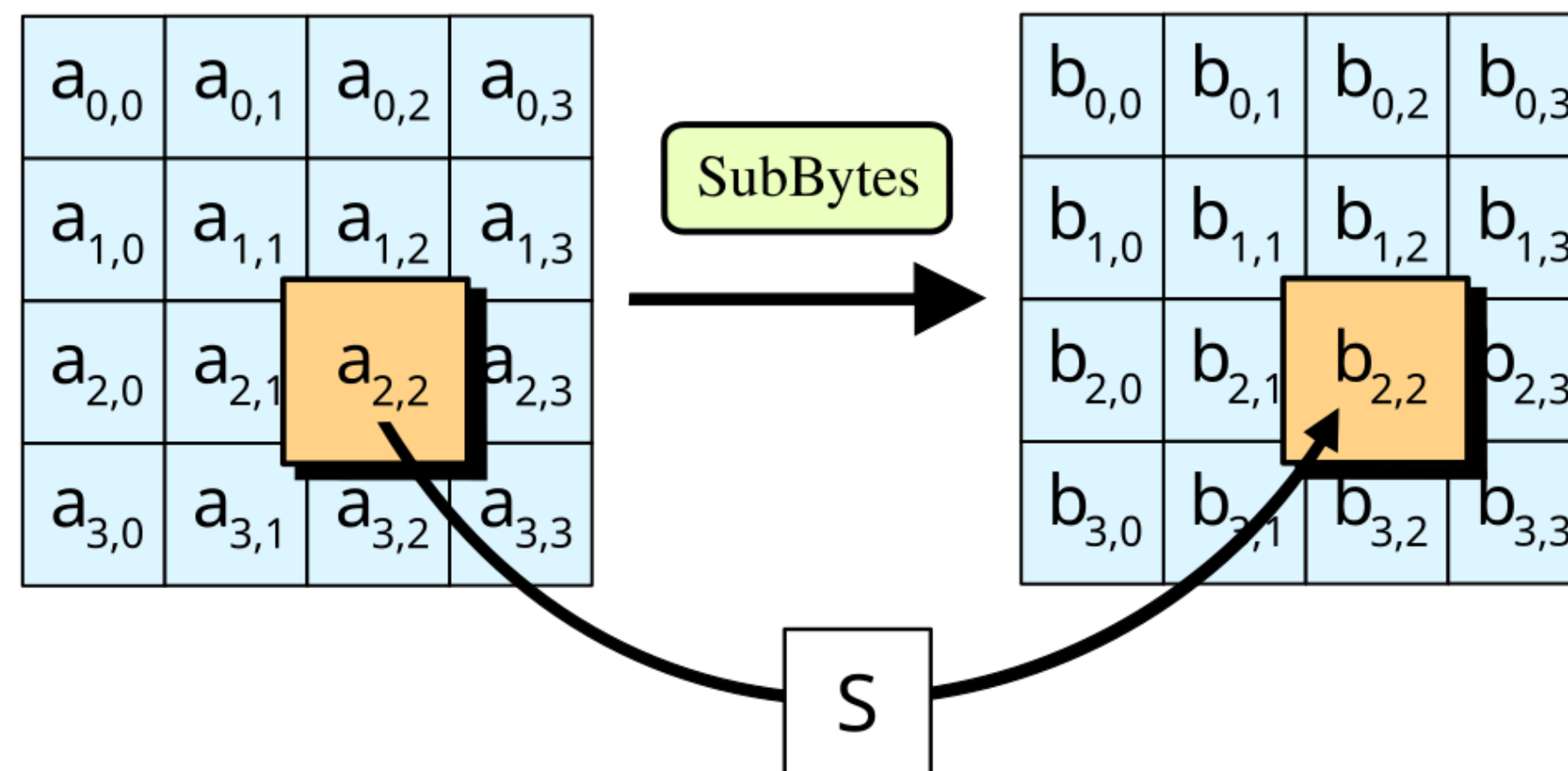


- Um bloco chamado **BO**, que serve como “*top-level*” para dois sub-blocos.
- Um bloco chamado **BO-KeyExpansion**, que trabalha exclusivamente no cálculo de todas as chaves de rodada simultaneamente e as registra.
- Um bloco chamado **BO-Criptografia**, que executa os blocos criptográficos, seleciona a chave de rodada advinda de KeyExpansion e registra o “*State*” em `regPC`. É deste registrador que o resultado final “*cipher_text*” advém, dependendo do round atual.

OBS.: Clock foi omitido de blocos síncronos a fim de melhor visualização do bloco, conforme orientado/baseado nos slides de aulas teóricas.

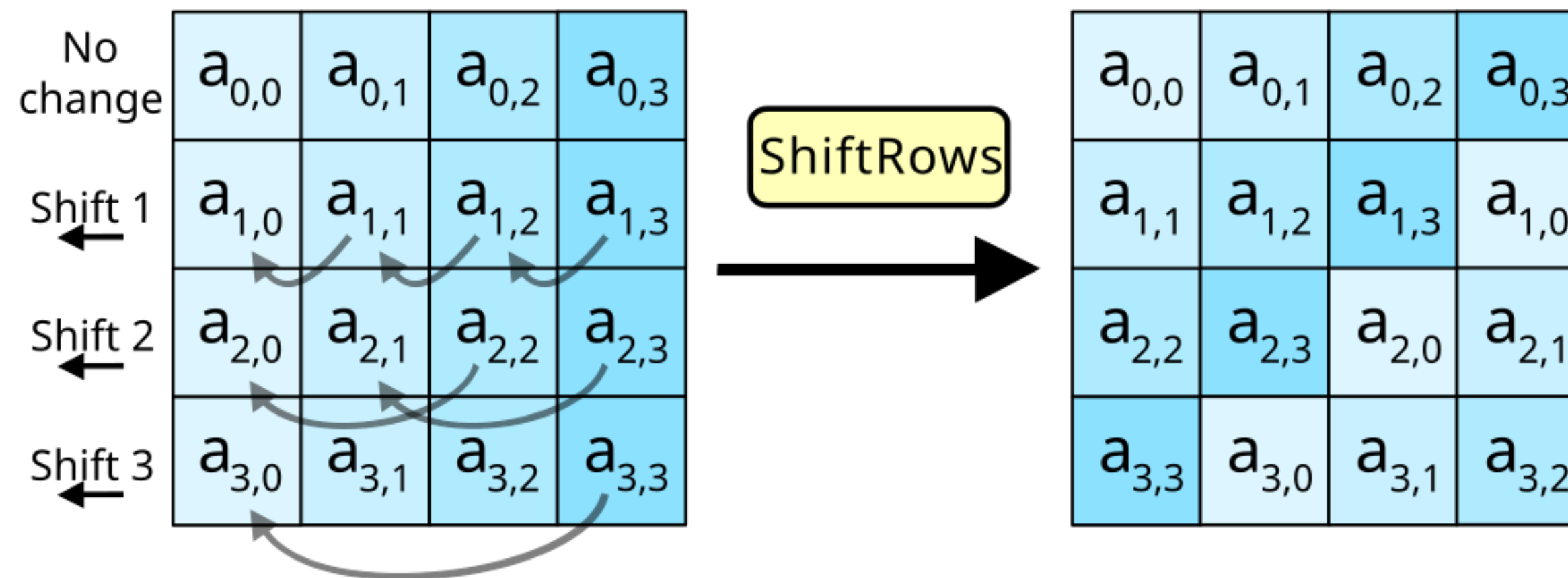
SubBytes

A operação SubBytes é a principal etapa responsável por causar confusão e quebrar a linearidade dentro do AES. Ela recebe a matriz 4x4 do Estado (State) e substitui cada um de seus bytes de forma independente através de uma tabela de busca conhecida como S-Box, gerando um novo valor.



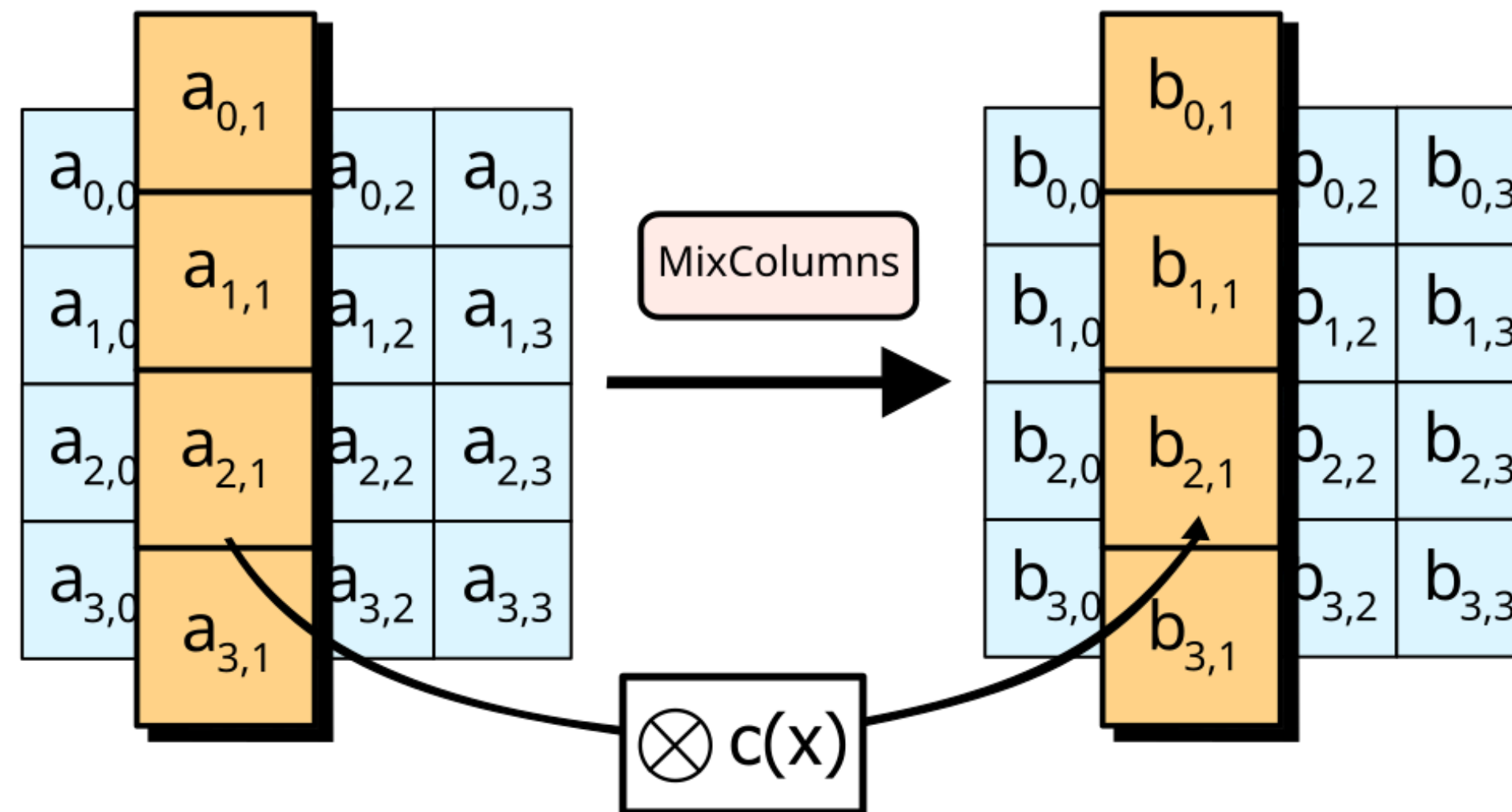
ShiftRows

O ShiftRows é o responsável por “embaralhar” os dados do bloco, a primeira linha permanece intacta, a segunda linha desloca os bytes um bytes a esquerda, a terceira linha desloca dois e a quarta linha desloca três.



MixColumns

Operação linear entre as colunas de dados. Ele multiplica e soma as matrizes bit a bit para gerar resultados. Os dados podem ser representados por uma matriz 4x1 e multiplicados por uma matriz 4x4 padrão, assim chegando a matriz resultado de 4x1 bytes. As operações são realizadas em $GF(2^8)$, onde a adição equivale a XOR e a multiplicação é feita módulo o polinômio irreduzível $x^8 + x^4 + x^3 + x + 1$, garantindo que o resultado sempre caiba em 8 bits. A matriz fixa $c(x)$ contém os coeficientes {01, 02, 03}, escolhidos para garantir boa difusão e uma implementação eficiente em hardware.

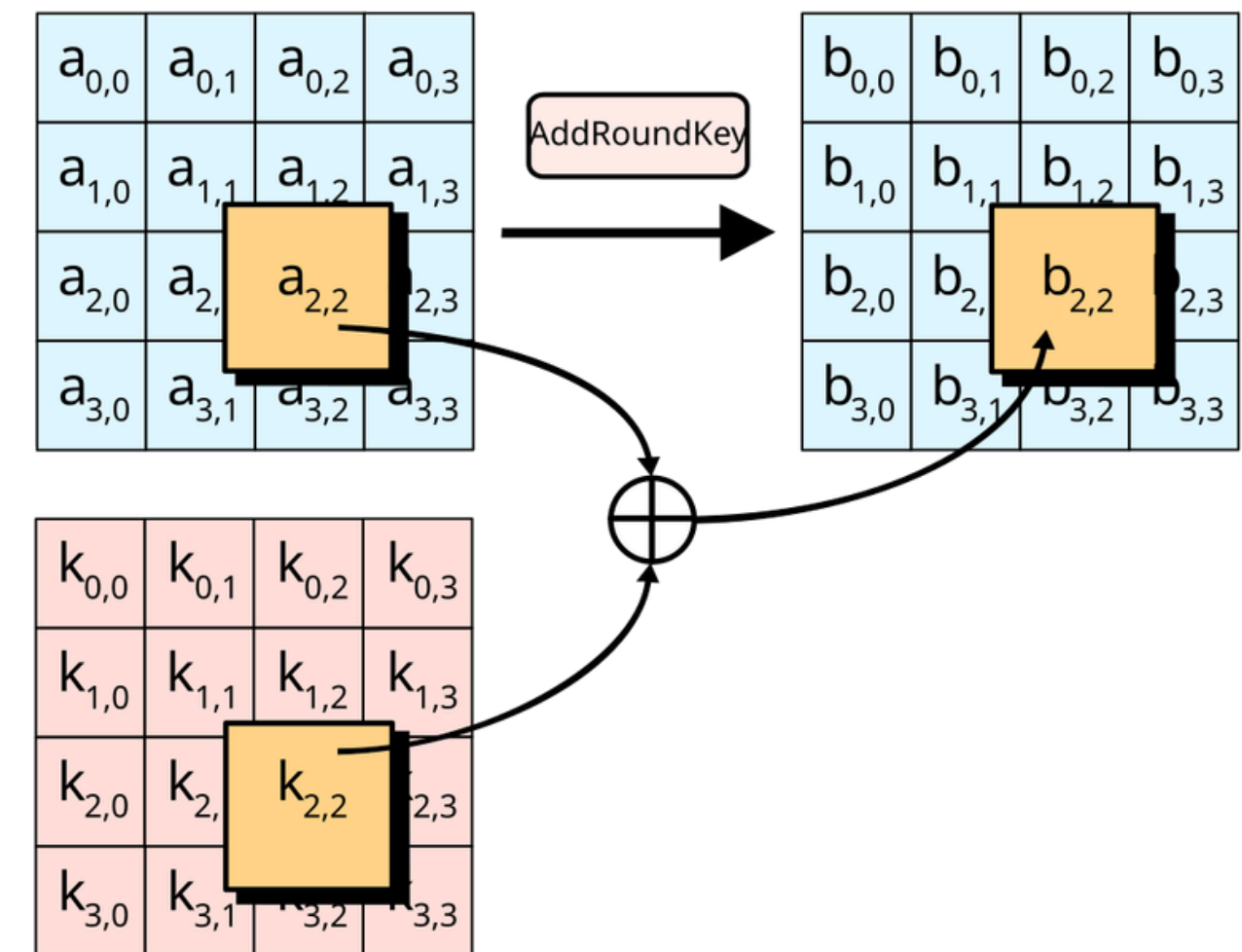


AddRoundKey

A operação AddRoundKey é a etapa do AES onde a subchave da rodada atual, gerada pelo KeyExpansion, é combinada com o estado atual dos dados (State). Esta é a principal operação de segurança do algoritmo, pois é o momento em que a chave secreta é injetada no texto claro.

A operação possui as seguintes características:

- **XOR:** operação Xor bit a bit com parte da chave estendida entregue pelo KeySchedule.
- **Matriz por Matriz:** Tanto o Estado quanto a subchave são tratados como matrizes 4×4 de bytes, declarado em AES_PACK





KeyExpansion

O componente KeyExpansion é responsável por realizar a "expansão" da chave original (de 128, 192 ou 256 bits) em várias sub-chaves menores, que serão utilizadas individualmente em cada rodada (round) do algoritmo AES.

Para gerar as subchaves, o KeySchedule aplica quatro transformações principais sobre as palavras (words) da chave:

- **RotWord(Rotação):** Pega os 4 bytes de uma palavra chave e descola um a esquerda.
- **SubWord(Substituição):** Aplica cada byte da palavra rotacionada à caixa de substituição (S-Box)..
- **Rcon(Constante de rodada):** Realiza uma operação lógica XOR entre o primeiro byte da palavra e uma constante matemática pré-definida de cada rodada, eliminando simetrias nas chaves geradas.
- **XOR com Palavras Anteriores:** Por fim, a palavra resultante desse processo passa por operações de XOR sequenciais com as palavras correspondentes da subchave anterior para gerar o bloco final da nova chave.



TestBenchs(AddRoundKey)

Exemplo de verificação da testbench:

```
assert teste out matriz(0, 0) = x"01" report "Falha Parte 2 - XOR Errado pos (0,0)" severity error;
```

Execução da testbench:

```
C:\Users\joaop\testes_testbench\05235A-Grupo-A-vhdl>ghdl -r --workdir=work --std=08 tb_addRoundKey --wave=tb_addRoundKey.gnw  
tb/tb_addRoundKey.vhdl:54:9:@20ns:(assertion error): Falha Parte 2 - XOR Errado pos (1,0)  
tb/tb_addRoundKey.vhdl:63:9:@20ns:(report note): Fim do teste addRoundKey. Se nao houver mensagens de erro acima, os testes PASSARAM com sucesso!
```

Erro de digitação nessa parte da testbench acarretou em erro:

```
assert teste out matriz(1, 0) = x"35" report "Falha Parte 2 - XOR Errado pos (1,0)" severity error;
```

TestBenchs(KeyExpansion)

Exemplo de verificação da testbench:

```
got := extract_round_key(s_round_keys, round);
if got = expected then
    report "PASS  " & test_label & " Round " & integer'image(round)
        severity note;
    pass_cnt := pass_cnt + 1;
else
    report "FAIL  " & test_label & " Round " & integer'image(round)
        & " expected=" & to_hstring(expected)
        & " got="      & to_hstring(got)
        severity error;
    fail_cnt := fail_cnt + 1;
end if;
```

Execução da testbench:

```
tb/tb_keyExpansion.vhdl:214:9:@50ns:(report note): RESULT : 39 passed, 0 failed (total = 39)
tb/tb_keyExpansion.vhdl:218:13:@50ns:(report note): ALL 39 ROUND KEYS VERIFIED CORRECTLY
tb/tb_keyExpansion.vhdl:223:9:@50ns:(report note): =====
```




TestBenchs(MixColumns)

Exemplo de verificação da testbench:

```
assert out matriz(0, 0) = x"04" report
```

Execução da testbench:

```
C:\Users\joaop\testes_testbench\05235A-Grupo-A-vhdl>ghdl -r --workdir=work --std=08 tb_mixColumns --wave=tb_mixColumns.ghw  
tb/tb_mixColumns.vhdl:92:9:@20ns:(report note): Fim do teste MixColumns. Se não houver mensagens de erro acima, o teste PASSOU com sucesso!
```



TestBenchs(ShiftRows)

Exemplo de verificação da testbench:

```
assert teste_out_matriz(0, 0) = x"10" report "Erro na pos (0,0)
```

Execução da testbench:

```
C:\Users\joaop\testes_testbench\05235A-Grupo-A-vhdl>ghdl -r --workdir=work --std=08 tb_shiftrows --wave=tb_shiftrows.ghw  
tb/tb_shiftrows.vhdl:78:9:@20ns:(report note): Fim do teste ShiftRows. Se não houver mensagens de erro acima, o teste PASSOU com sucesso!
```



TestBenchs(SubBytes)

Exemplo de verificação da testbench:

```
for i in 0 to 3 loop
  for j in 0 to 3 loop

    valor := base + i*4 + j;

    assert matriz_out_tb(i,j) = SBOX(valor)
    report "Erro para valor decimal: " & integer'image(valor)
    severity error;

  end loop;
end loop;
```

Execução da testbench:

```
tb/tb_subBytes.vhdl:68:13:@160ns:(report note): Bloco 15 aprovado
tb/tb_subBytes.vhdl:72:9:@160ns:(report note): TODOS OS 256 TESTES PASSARAM
```



TestBenchs(TopLevel)

Comparação dos resultados obtidos com a tabela Fips:

Vetor de teste oficial do AES-128:

Key: 000102030405060708090a0b0c0d0e0f

user_text: 00112233445566778899aabbccddeeff

ciphertext: 69c4e0d86a7b0430d8cdb78070b4c55a

000102030405060708090A0B0C0D0E0F0			
00112233445566778899AABBCCDDEEFF			
69C4E0D86A7B0430D8CDB78070B4C55A			

Vetor de teste oficial do AES-192:

Key: 000102030405060708090a0b0c0d0e0f1011121314151617

user_text: 00112233445566778899aabbccddeeff

ciphertext: dda97ca4864cdf06eaf70a0ec0d7191

000102030405060708090A0B0C0D0E0F1011121314151617		
00112233445566778899AABBCCDDEEFF		
DDA97CA4864CDFE06EAF70A0EC0D7191		

Vetor de teste oficial do AES-256:

Key: 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f

user_text: 00112233445566778899aabbccddeeff

ciphertext: 8ea2b7ca516745bfeafc49904b496089

000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F			
00112233445566778899AABBCCDDEEFF			
8EA2B7CA516745BFEAFC49904B496089			

TestBenchs(TopLevel)

Existe uma matriz assim para cada tipo de AES

```
type expected_array_128 is array(0 to 10) of std_logic_vector(127 downto 0);
constant FIPS_EXPECTED_128 : expected_array_128 := (
    0 => x"00102030405060708090a0b0c0d0e0f0", -- Start of Round 1
    1 => x"89d810e8855ace682d1843d8cb128fe4", -- Start of Round 2
    2 => x"4915598f55e5d7a0daca94fa1f0a63f7", -- Start of Round 3
    3 => x"fa636a2825b339c940668a3157244d17", -- Start of Round 4
    4 => x"247240236966b3fa6ed2753288425b6c", -- Start of Round 5
    5 => x"c81677bc9b7ac93b25027992b0261996", -- Start of Round 6
    6 => x"c62fe109f75eedc3cc79395d84f9cf5d", -- Start of Round 7
    7 => x"d1876c0f79c4300ab45594add66ff41f", -- Start of Round 8
    8 => x"fde3bad205e5d0d73547964ef1fe37f1", -- Start of Round 9
    9 => x"bd6e7c3df2b5779e0b61216e8b10b689", -- Start of Round 10
    10 => x"69c4e0d86a7b0430d8cdb78070b4c55a" -- Output Final
);
```




Exemplo de verificação da testbench:

TestBenchs(TopLevel)

Testbench AES-128

```
for i in 0 to 10 loop
  wait for HOLD_TIME;

  assert ciphertext = FIPS_EXPECTED_128(i)
    report "AES-128 FALHA CRITICA NA RODADA " & integer'image(i+1) &
      " | Esperado: " & to_hstring(FIPS_EXPECTED_128(i)) &
      " | Obtido: " & to_hstring(ciphertext)
    severity error;

  if i < 10 then
    wait until falling_edge(clk);
  end if;
end loop;
```

Testbench AES-192

```
for i in 0 to 12 loop
  wait for HOLD_TIME;

  assert ciphertext = FIPS_EXPECTED_192(i)
    report "AES-192 FALHA CRITICA NA RODADA " & integer'image(i+1) &
      " | Esperado: " & to_hstring(FIPS_EXPECTED_192(i)) &
      " | Obtido: " & to_hstring(ciphertext)
    severity error;

  if i < 12 then
    wait until falling_edge(clk);
  end if;
end loop;
```

Testbench AES-256

```
for i in 0 to 14 loop
  wait for HOLD_TIME;

  assert ciphertext = FIPS_EXPECTED_256(i)
    report "AES-256 FALHA CRITICA NA RODADA " & integer'image(i+1) &
      " | Esperado: " & to_hstring(FIPS_EXPECTED_256(i)) &
      " | Obtido: " & to_hstring(ciphertext)
    severity error;

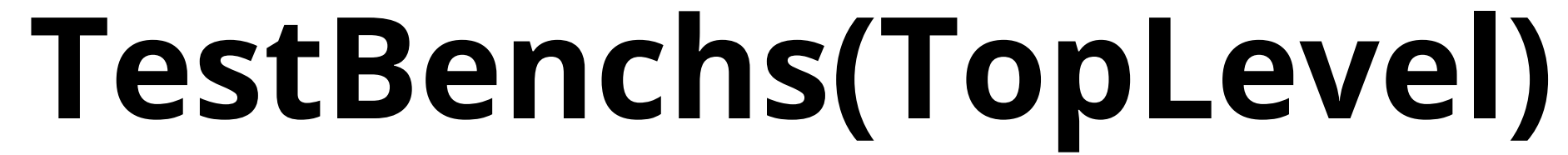
  if i < 14 then
    wait until falling_edge(clk);
  end if;
end loop;
```



TestBenchs(TopLevel)

Execução da testbench:

```
VSIM 14> # 7. Execute a simulação
VSIM 15> run 10 us
# ** Note: -----
#   Time: 0 ps   Iteration: 0   Instance: /tb_top_level
# ** Note: INICIANDO TESTE DETALHADO (AES-128 FIPS)
#   Time: 0 ps   Iteration: 0   Instance: /tb_top_level
# ** Note: -----
#   Time: 0 ps   Iteration: 0   Instance: /tb_top_level
# ** Note: Teste Detalhado AES-128 PASSOU com sucesso!
#   Time: 284 ns  Iteration: 0   Instance: /tb_top_level
# ** Note: -----
#   Time: 290 ns  Iteration: 0   Instance: /tb_top_level
# ** Note: INICIANDO TESTE DETALHADO (AES-192 FIPS)
#   Time: 290 ns  Iteration: 0   Instance: /tb_top_level
# ** Note: -----
#   Time: 290 ns  Iteration: 0   Instance: /tb_top_level
# ** Note: Teste Detalhado AES-192 PASSOU com sucesso!
#   Time: 624 ns  Iteration: 0   Instance: /tb_top_level
# ** Note: -----
#   Time: 630 ns  Iteration: 0   Instance: /tb_top_level
# ** Note: INICIANDO TESTE DETALHADO (AES-256 FIPS)
#   Time: 630 ns  Iteration: 0   Instance: /tb_top_level
# ** Note: -----
#   Time: 630 ns  Iteration: 0   Instance: /tb_top_level
# ** Note: Teste Detalhado AES-256 PASSOU com sucesso!
#   Time: 1004 ns Iteration: 0   Instance: /tb top level
```

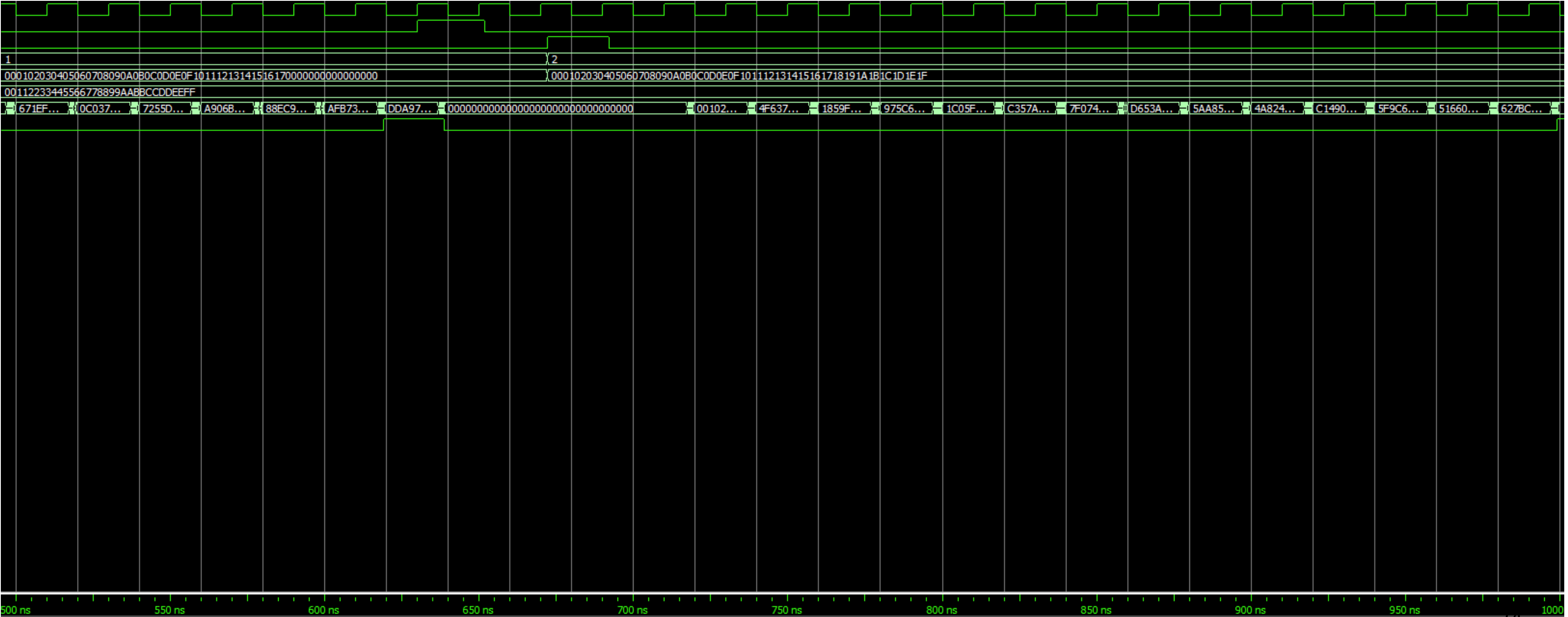



Msgs																													
/tb_top_level/dk	0																												
/tb_top_level/rst_a	1																												
/tb_top_level/init	0																												
/tb_top_level/aes_t...	2'h0																												
/tb_top_level/user_...	256'h000000000...																												
/tb_top_level/user_...	128'h000000000...																												
/tb_top_level/ciphe...	128'h000000000...																												
/tb_top_level/done	0																												
Now	10000 ns																												
Cursor 1	0.00 ns																												



TestBenchs(TopLevel)

Waves com atraso real:





Resultados

Resultados

- O coprocessador atingiu uma frequência máxima de operação de 125,63 MHz, valor considerado altamente satisfatório para a família Cyclone IV E.
- O número de pinos é resultado da soma de larguras do top-level: $1 \cdot \text{clk} + 1 \cdot \text{rst_a} + 1 \cdot \text{init} + 128 \cdot \text{user_text} + 256 \cdot \text{user_key} + 2 \cdot \text{aes_type} + 1 \cdot \text{done} + 128 \cdot \text{cipher_text} = 518$.
- O cálculo de todas as chaves de rodada de uma única vez, que opera com 1.920 bits, é responsável por elevar o número de registradores lógicos dedicados.
- O elevado número de funções combinacionais decorre, principalmente, da estratégia de mapear as tabelas de busca (S-Boxes) como lógica combinacional pura, devido ao acesso massivo e simultâneo do algoritmo aos dados de uma matriz de 1.785 bits, várias vezes.

Metрика de Hardware	Valor Obtido
Frequência Máxima	125,63 MHz
Pinos Totais	518
Registradores lógicos dedicados	2.056
Funções combinacionais totais	36.005

Resultados

Registradores Lógicos Dedicados

Uma possível reformulação a fim de diminuir o número de registradores seria o cálculo das chaves de rodada *"on-the-fly"* - rodada a rodada. Entretanto, isso impossibilitaria o reuso do código para um algoritmo de descryptografia, pois ele exige o cálculo de uma só vez. Além disso, este número ainda sim não seria menor do que 512, pois é o mínimo de bits gastos pelo registro de pelo menos 8 Words, necessárias para o cálculo da chave de rodada atual no AES-256. Outrossim, este tipo de cálculo aumenta significativamente a complexidade da implementação.

Funções combinacionais totais

O método para diminuição do número de funções combinacionais seria a instanciação da S-BOX via memória RAM. Entretanto, o ideal é que isso fosse fundamentalizado na concepção do projeto, visto que uma reformulação posterior geraria os seguintes percalços:

- Faria-se necessário reformular a FSMD por completo. O estado Scalc seria substituído por um estado para cada etapa do cálculo, pois o acesso a RAM é síncrono;
- A S-BOX é acessada simultaneamente 52 vezes no keyExpansion. No cenário mais otimizado, utilizaria-se pelo menos 20,6% da memória disponível (52 inst. de S-box);
- O uso de *functions* não seria possível em alguns casos, como em subBytes e subWords (keySchedule), pois funções não conseguem ter acesso a RAM. Portanto, blocos de cálculo seriam reformulados quase que por completo.



Conclusão

Foi apresentado o projeto, modelagem e implementação em hardware de um coprocessador criptográfico baseado no algoritmo AES, desenvolvido em VHDL e validado na FPGA Cyclone IV E. O grande diferencial do projeto foi a inclusão de uma chave seletora externa (AESType), que possibilita a escolha dinâmica, em tempo de execução, entre as variantes AES-128, AES-192 e AES-256, realizando cifragem e decifragem com total precisão matemática.

Os resultados evidenciaram uma ponderação entre área e flexibilidade. Embora o mapeamento das S-Boxes em lógica combinacional pura tenha expandido o circuito para 36.005 funções combinacionais e elevado o tempo de compilação, a arquitetura paralela garantiu uma frequência máxima satisfatória de 125,63 MHz, com integridade validada via simulações no GHDL e QuestaSim. Como trabalhos futuros, sugere-se reformular a FSMD para transformar as sub-rotinas do pacote AES_PACK (como a SubWord) em entidades estruturais. Isso viabilizará o acesso direto às memórias RAM nativas da FPGA (M9K), permitindo avaliar a redução da área combinatória em contraste com o aumento no número de ciclos de clock.



Bibliografia

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). Federal Information Processing Standards Publication 197 (FIPS 197): Advanced Encryption Standard (AES). Gaithersburg, MD: NIST, 26 nov. 2001. Disponível em: <https://csrc.nist.gov/pubs/fips/197/final>.

CAMPOS, Alessandro Augusto Nunes. Algoritmo de criptografia AES em hardware, utilizando Dispositivos de Lógica Programável (FPGA) e Linguagem de Descrição de Hardware (VHDL). 2008. Dissertação (Mestrado em Engenharia Elétrica) – Universidade Federal de Itajubá, Itajubá, 2008. Disponível em: <https://repositorio.unifei.edu.br/xmlui/handle/123456789/1693>.

VECCHIATO, Daniel Avila. Criptografia e Segurança de Dados - AES (Advanced Encryption Standard). [S. l.: s. n.], 9 out. 2020. 1 vídeo (31 min). Publicado pelo canal Daniel Vecchiato. Disponível em: <https://www.youtube.com/watch?v=-lybDqNi-bM>.

GOOGLE. Gemini. Mountain View: Google, 2026. Disponível em: <https://gemini.google.com>.

CANVA. Canva. Sydney: Canva, 2026. Disponível em: <https://www.canva.com/>.

JGRAPH. draw.io (diagrams.net). Londres: JGraph, 2026. Disponível em: <https://app.diagrams.net/>.